

● PART 1: COMPREHENSIVE SOFTWARE FEATURE BREAKDOWN

The system is split between the **APMS Dashboard** (for the pharmacy staff/managers) and **Roshetty** (the patient-facing mobile app). Both communicate via a centralized Node.js/PostgreSQL gateway.

1. Core Pharmacy Operations (APMS)

- **Role-Based Access Control (JWT):** The system distinguishes between **Manager** and **Staff** roles. Managers get full access to AI models, financial reports, and user creation, while staff are restricted to operational scanning and dispensing.
- **Real-Time Dashboard:** Displays top-level KPIs (Revenue, Pending Prescriptions, Expiring Items). Features a toggleable line chart for 7-day or 7-month cash flow and live inventory alerts.
- **Inventory Management:**
 - Real-time stock tracking with color-coded health bars.
 - Filters for low stock, expiring items, and refrigerated (cold-chain) medicines.
 - **Medicine Form:** A detailed data entry modal that calculates gross profit margins dynamically based on input cost and selling price.
 - **Excel Import:** A robust, crash-protected Excel importer (**xlsx**) to bulk-upload thousands of medicines instantly.
- **Prescription Lifecycle:**
 - Tracks orders through **PENDING**, **PARTIALLY_DISPENSED**, **COMPLETE**, and **CANCELLED** states.
 - **Checkout Logic:** Automatically deducts physical inventory and locks in the historical **pay_rate_at_sale** to ensure financial reports remain accurate even if medicine prices change later.
- **Invoicing & Fawateer:**
 - Manages supplier purchases.
 - *Smart Stock Update:* Inventory quantities and unit costs are *only* updated in the database when an invoice status is manually flipped to **PAID**.

2. Artificial Intelligence & OCR

- **AI Command Center:** Connects to a Python backend running predictive models.
 - **Demand Forecasting (XGBoost):** Predicts next month's total unit sales based on historical Excel data.
 - **Smart Reorder (Q-Learning):** Recommends exactly how many packs to order based on standard deviation and average monthly velocity.
 - **Integrity Audit (Autoencoder):** Detects financial anomalies in shift reconciliations (e.g., cash drawer missing more than 300 EGP).
 - **Expiry Risk & Clearance:** Flags expired items for immediate removal and suggests dynamic discount percentages (up to 50%) for items nearing expiry.
- **Computer Vision (OCR):**
 - Reads physical Roshettas or supplier invoices via camera/gallery.

- Utilizes a two-pass fuzzy search algorithm to match messy, misspelled OCR text against the clean PostgreSQL database inventory.

3. Roshetty Patient App

- **Bilingual & Accessible:** Seamless toggle between English and Arabic (LTR/RTL) with a soothing, pastel aesthetic to minimize patient anxiety.
- **Remote Ordering:** Patients can type a medicine name manually or snap a picture of their prescription to send directly to the APMS **PENDING** queue via the public `/api/public/check-prescription` endpoint.

● PART 2: THE HARDWARE INTEGRATION BIBLE (FOR YOUR TEAM)

ATTENTION HARDWARE ENGINEERS: The Node.js backend currently *fakes* the robot's movement using JavaScript timers (`setTimeout`) so the frontend UI could be designed. **You must rip this simulation out and replace it with your physical microcontroller logic.**

Here is the exact, step-by-step workflow for integrating the ESP32/Raspberry Pi.

STEP 1: Understand the WebSocket Connection

The APMS Node.js server hosts a WebSocket feed on port **4000** (`index.ts`). Your hardware must maintain a persistent connection to this socket:

- **URL:** `ws://<LOCAL_PC_IP>:4000`
- **Why?** This is how the hardware listens for instant commands from the iPad without constantly pinging the server.

STEP 2: Ripping out the "Fake" Robot

Open `express_routes/robot.ts` and locate `router.post('/dispatch', ...)` and `router.post('/retry/:jobId', ...)`. Currently, the code looks like this:

TypeScript

```
setTimeout(() => {
  broadcastEvent({ type: 'ROBOT_ACK', jobId, status: 'DISPATCHED', message: 'Robot has acknowledged job' });

  setTimeout(async () => {
    // ... database inventory deduction and auto-checkout ...
    await query(`UPDATE robot_jobs SET status = $1 ...`);
  }, 5000); // <-- THIS IS THE FAKE 5 SECOND ROBOT MOVEMENT
}, 1000);
```

Your Job: Delete that inner 5000ms `setTimeout`. The `POST /dispatch` route should *only* insert the job into the database and broadcast the `DISPATCHED` event. The physical robot will dictate when the job is actually done.

STEP 3: The Execution Flow (What the Robot Must Do)

1. **Listen for the Wake-Up Call:** When the Product Engineer taps "Dispatch Robot" on the UI, the Node server broadcasts this over the WebSocket:
2. JSON

```
{ "type": "ROBOT_ACK", "jobId": "JOB-1714681200", "status": "DISPATCHED" }
```

- 3.
- 4.
5. **Fetch the Pick Sequence:** The WebSocket payload only tells the robot a job exists. The hardware must now make an HTTP GET request to fetch the physical coordinates.
 - **Endpoint:** `GET`
`http://<LOCAL_PC_IP>:4000/api/robot/jobs/JOB-1714681200`
 - **The Target Data:** Look inside the `pick_sequence` array:
 - JSON

```
[  
  { "medicine_id": 105, "qty": 2, "bin": "A-01-A", "status": "PENDING" }  
]
```

- -
6. **Move the Motors:** Parse the `bin` string (e.g., `A-01-A`). Your C++/Python firmware must map this string to physical X/Y/Z stepper motor steps. Drop the exact `qty` specified.
 7. **Report Success Back to Node.js (CRITICAL):** Once the items hit the dispensing tray, the hardware *must* tell the backend it is finished so the UI updates and the inventory is deducted.
 - **You need to build a new route** in `robot.ts` (e.g., `POST /api/robot/jobs/:jobId/complete`).
 - When the hardware calls this route, the backend must update the `robot_jobs` table to `COMPLETED`, deduct the inventory from `medicines_inventory`, and broadcast a `COMPLETED` WebSocket event so the Flutter UI turns green.

STEP 4: Handling Emergency Aborts

The UI has a red "Stop Arm" button. If the pharmacist presses it, the Node server broadcasts:

JSON

```
{ "type": "ROBOT_ACK", "jobId": "JOB-123", "status": "ABORTED" }
```

Your microcontroller must have an Interrupt Service Routine (ISR) constantly listening to the WebSocket for the "ABORTED" status. If received, it must instantly cut the signal to the stepper drivers and execute a safe return-to-home sequence.